

Lecture 6: reliability

documenting protocols and classes, concurrent invariants, assertions, stress testing, estimating bug probability, static and dynamic checks, scheduling randomization and determinisation, model checking

Alexander Filatov
filatovaur@gmail.com

<https://github.com/Svazars/parallel-programming/blob/main/slides/pdf/l6.pdf>

In previous episodes

Concurrent coordination concepts:

- Mutual exclusion, Signalling, Group-level concurrency, Separation of threads and tasks
- Cancellation, interruption

Concurrent data structures:

- Lock, Condition, Monitor, CountdownLatch, Semaphore, ReadWriteLock

Concurrent programming patterns:

- Code locking, Data locking, Lock splitting, Exponential backoff
- Replication, Snapshotting, Concurrent state machine

Concurrent software problems:

- Race condition, data race, deadlock
- Lost signal, predicate invalidation, spurious wakeup, fairness
- Thundering herd, oversubscription

Pain points

- Unpredictable speed of execution
- Arbitrary "whole program control flow"
 - `Thread.start` looks like `goto`
- Arbitrary "thread-specific control flow"
 - timeout, notification, interruption, spurious wakeup
- Limited composability of modules
 - locking policy is not clear
 - lock ordering hidden by virtual methods
 - exposed `synchronized`
 - non-documented thread-safety
- Hard to diagnose performance problems
 - underutilization, priority inversion, thundering herd, scalability
- It is **too easy** to introduce a race condition
 - it is **too hard** to find it on review, during testing, debug on production system

Question time

Question: Concurrency is a very complicated programming domain. What could we do to simplify our programming life?



Lecture plan

- 1 Concurrent invariants
- 2 Testing
 - Unit testing
 - Stress testing
 - Bug probability estimation
- 3 Tooling
- 4 Scheduling control
- 5 Bounded model checking
- 6 Summary

Lecture plan

- 1 Concurrent invariants
- 2 Testing
 - Unit testing
 - Stress testing
 - Bug probability estimation
- 3 Tooling
- 4 Scheduling control
- 5 Bounded model checking
- 6 Summary

Question time

Question: How user of your program will know its features and limitations?



Documentation

Reading the docs:

- Thread-safety of a class
- How to use class to avoid inconsistent state (e.g. `try-finally` for `Lock`)
- Blocking operations
- Locking policy
- Admission policy
- Inheritance policy

Documentation

Reading the docs:

- Thread-safety of a class
- How to use class to avoid inconsistent state (e.g. `try-finally` for `Lock`)
- Blocking operations
- Locking policy
- Admission policy
- Inheritance policy

Writing the docs:

Documentation

Reading and **writing** the docs:

- Thread-safety of a class
- How to use class to avoid inconsistent state (e.g. `try-finally` for `Lock`)
- Blocking operations
- Locking policy
- Admission policy
- Inheritance policy

Documentation

Reading and **writing** the docs:

- Thread-safety of a class
- How to use class to avoid inconsistent state (e.g. try-finally for Lock)
- Blocking operations
- Locking policy
- Admission policy
- Inheritance policy

`java.util.concurrent` is a good source for inspiration

Question time

Question: Nobody reads the documentation! How could I **enforce** usage patterns of my class?



Invariants checking

Single-threaded code crashes with `ConcurrentModificationException`¹:

```
void foo(List<X> list) {  
    for (X x : list) {  
        if (!x.isValid()) list.remove(x);  
    }  
}
```

¹<https://stackoverflow.com/questions/3184883/concurrentmodificationexception-for-arraylist>

Invariants checking

Single-threaded code crashes with `ConcurrentModificationException`¹:

```
void foo(List<X> list) {
    for (X x : list) {
        if (!x.isValid()) list.remove(x);
    }
}
```

It is intended behaviour²

The iterators returned by this class's 'iterator' ... are fail-fast: if the list is structurally modified at any time after the iterator is created, in any way except through the iterator's own 'remove' or 'add' methods, the iterator will throw a 'ConcurrentModificationException'. Thus, in the face of concurrent modification, the iterator fails quickly and cleanly, rather than risking arbitrary, non-deterministic behavior at an undetermined time in the future.

¹ <https://stackoverflow.com/questions/3184883/concurrentmodificationexception-for-arraylist>

² <https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/ArrayList.html>

Question time

Question: How to implement such consistency check (collection is not modified while iterated)?



Invariants checking

Advantages:

- Contract is enforced by implementation rather by javadoc
- Fail-fast behaviour speed-ups debugging
- Type of exception and stacktrace information help to debug large systems

Disadvantages:

- Performance overheads
- Incompleteness (impossible to cover all misuses)
- Affects even single-threaded programs

Question time

Question: How could we implement lightweight or zero-cost checks?



Assertions

Assertions³:

```
void foo(ArrayList<String> list) {  
    assert list != null;  
    assert list.size() > 0;  
}
```

³ <https://docs.oracle.com/javase/specs/jls/se11/html/jls-14.html#jls-14.10>

Assertions

Assertions³:

```
void foo(ArrayList<String> list) {  
    assert list != null;  
    assert list.size() > 0;  
}
```

Typically, assertion checking is enabled during program development and testing, and disabled for deployment, to improve performance.

³ <https://docs.oracle.com/javase/specs/jls/se11/html/jls-14.html#jls-14.10>

Assertions

Assertions³:

```
void foo(ArrayList<String> list) {  
    assert list != null;  
    assert list.size() > 0;  
}
```

Typically, assertion checking is enabled during program development and testing, and disabled for deployment, to improve performance.

Because assertions may be disabled, programs must not assume that the expressions contained in assertions will be evaluated. Thus, these boolean expressions should generally be free of side effects.

³ <https://docs.oracle.com/javase/specs/jls/se11/html/jls-14.html#jls-14.10>

Assertions

Assertions³:

```
void foo(ArrayList<String> list) {  
    assert list != null;  
    assert list.size() > 0;  
}
```

Typically, assertion checking is enabled during program development and testing, and disabled for deployment, to improve performance.

Because assertions may be disabled, programs must not assume that the expressions contained in assertions will be evaluated. Thus, these boolean expressions should generally be free of side effects.

In light of this, assertions should not be used for argument checking in public methods. Argument checking is typically part of the contract of a method, and this contract must be upheld whether assertions are enabled or disabled.

³ <https://docs.oracle.com/javase/specs/jls/se11/html/jls-14.html#jls-14.10>

Assertions

- Could be enabled/disabled without recompilation of Java program⁴
 - Many guidelines enforce `java -ea -esa -jar production.jar ...`
- Should be used for checking **internal consistency** (violation of programmer's intent), not **external consistency** (e.g validation of user input)
- Perfect choice to make concurrent programs fail-fast
 - Use `-enablesystemassertions` to fail-fast on subtle data races inside improperly used classes from standard library (e.g. unsynchronized access to `ArrayList`)

⁴ <https://docs.oracle.com/javase/8/docs/technotes/guides/language/assert.html>

Optional heavyweight checks

```
class MyLinkedList {  
    private int cachedLength;  
    private Node head;  
    private int length() {  
        int size = 0;  
        for (Node n = this.head; n != null; n = n.next, size++) {}  
        return size;  
    }  
    public void add(Node e) {  
        e.next = this.head;  
        this.head = e;  
        cachedLength++;  
        assert cachedLength == length();  
    }  
}
```

Optional heavyweight checks

```
class MyLinkedList {  
    private int cachedLength;  
    private Node head;  
    private int length() {  
        int size = 0;  
        for (Node n = this.head; n != null; n = n.next, size++) {}  
        return size;  
    }  
    public void add(Node e) {  
        e.next = this.head;  
        this.head = e;  
        cachedLength++;  
        assert heavyChecksDisabled() || cachedLength == length();  
    }  
}
```

Optional heavyweight checks

- Could find non-local data inconsistency
- May be prohibitively slow for production workloads
- Distort concurrent execution patterns (change probability of race condition)
- May be designed to provide detailed information about failure

Summary: invariants

- Invariants help to make better contracts and design more reliable programs
- Report failed external contracts with detailed exceptions
- Check internal contracts with lightweight checks (e.g. optional asserts)
- Consider using heavyweight checks for special modes of execution (e.g. debug builds)

Summary: invariants

- Invariants help to make better contracts and design more reliable programs
- Report failed external contracts with detailed exceptions
- Check internal contracts with lightweight checks (e.g. optional asserts)
- Consider using heavyweight checks for special modes of execution (e.g. debug builds)

You can not start checking for correctness until you understand

- allowed
- forbidden

result of your algorithm or data structure.

Question time

Question: What are the main requirements for thread-safe code?



Concurrent invariants and thread safety

Some validity rules are obvious and trivial:

- Does not fail with `AssertionError`, `NullPointerException` ...
- Deadlock never happens, data race never happens ...
- ...

Concurrent invariants and thread safety

Some validity rules are obvious and trivial:

- Does not fail with `AssertionError`, `NullPointerException` ...
- Deadlock never happens, data race never happens ...
- ...

Describing behaviour in multithreaded environment requires additional effort:

- After data is added to thread-safe collection, some other thread eventually could remove it

Concurrent invariants and thread safety

Some validity rules are obvious and trivial:

- Does not fail with `AssertionError`, `NullPointerException` ...
- Deadlock never happens, data race never happens ...
- ...

Describing behaviour in multithreaded environment requires additional effort:

- After data is added to thread-safe collection, some other thread eventually could remove it
- only once

Concurrent invariants and thread safety

Some validity rules are obvious and trivial:

- Does not fail with `AssertionError`, `NullPointerException` ...
- Deadlock never happens, data race never happens ...
- ...

Describing behaviour in multithreaded environment requires additional effort:

- After data is added to thread-safe collection, some other thread eventually could remove it
- only once

Concurrent consistency could be hard to explain and check⁵:

... the result of any execution is the same as if the operations of all the processors were executed in some sequential order, and the operations of each individual processor appear in this sequence in the order specified by its program.

⁵ https://en.wikipedia.org/wiki/Sequential_consistency

Concurrent invariants and thread safety

Some validity rules are obvious and trivial:

- Does not fail with `AssertionError`, `NullPointerException` ...
- Deadlock never happens, data race never happens ...
- ...

Describing behaviour in multithreaded environment requires additional effort:

- After data is added to thread-safe collection, some other thread eventually could remove it
- only once

Concurrent consistency could be hard to explain and check⁵:

... the result of any execution is the same as if the operations of all the processors were executed in some sequential order, and the operations of each individual processor appear in this sequence in the order specified by its program.

Alarm bell №1: we do not know how to formalize *consistency*

⁵ https://en.wikipedia.org/wiki/Sequential_consistency

Question time

Question: How do you **automatically** check that your program does not fail with assertions?



Lecture plan

- 1 Concurrent invariants
- 2 Testing
 - Unit testing
 - Stress testing
 - Bug probability estimation
- 3 Tooling
- 4 Scheduling control
- 5 Bounded model checking
- 6 Summary

Unit testing

Focus on safety (correctness).

Unit testing

Focus on safety (correctness).

- All single-threaded scenarios
- Basic multi-threaded scenarios
- Advanced multi-threaded scenarios with custom scheduling (insertion of `Thread.sleep`)

Unit testing

Focus on safety (correctness).

- All single-threaded scenarios
- Basic multi-threaded scenarios
- Advanced multi-threaded scenarios with custom scheduling (insertion of `Thread.sleep`)

Goals:

- coverage
- regression tests
- supplementary documentation for intended and forbidden usages

Unit testing

Focus on safety (correctness).

- All single-threaded scenarios
- Basic multi-threaded scenarios
- Advanced multi-threaded scenarios with custom scheduling (insertion of `Thread.sleep`)

Goals:

- coverage
- regression tests
- supplementary documentation for intended and forbidden usages

Tools:

- JUnit⁶

⁶<https://junit.org>

Question time

Question: Unit tests are small and fast. How to ensure concurrent software properly works under pressure?



Lecture plan

- 1 Concurrent invariants
- 2 Testing
 - Unit testing
 - **Stress testing**
 - Bug probability estimation
- 3 Tooling
- 4 Scheduling control
- 5 Bounded model checking
- 6 Summary

JCStress

<https://github.com/openjdk/jcstress>

```
@JCStressTest
```

```
@Outcome(id = "1, 2", expect = ACCEPTABLE, desc = "actor1 -> actor2.")
```

```
@Outcome(id = "2, 1", expect = ACCEPTABLE, desc = "actor2 -> actor1.")
```

```
@State
```

```
public class TestConcurrentIncrements {
```

```
    int v;
```

```
    @Actor public void actor1(II_Result r) { r.r1 = ++v; }
```

```
    @Actor public void actor2(II_Result r) { r.r2 = ++v; }
```

```
}
```

JCStress

```

@JCStressTest
@Outcome(id = "1, 2", expect = ACCEPTABLE, desc = "actor1 -> actor2.")
@Outcome(id = "2, 1", expect = ACCEPTABLE, desc = "actor2 -> actor1.")
@State
public class TestConcurrentIncrements {
    int v;
    @Actor public void actor1(II_Result r) { r.r1 = ++v; }
    @Actor public void actor2(II_Result r) { r.r2 = ++v; }
}

```

RESULT	SAMPLES	FREQ	EXPECT	DESCRIPTION
1, 1	120,616	0.52%	Forbidden	No default case provided
1, 2	22,993,014	99.01%	Acceptable	actor1 -> actor2.
2, 1	108,661	0.47%	Acceptable	actor2 -> actor1.

JCStress

```
@JCStressTest
@Outcome(id = "1, 1", expect = ACCEPTABLE_INTERESTING, desc = "Data race")
@Outcome(id = "1, 2", expect = ACCEPTABLE, desc = "actor1 -> actor2.")
@Outcome(id = "2, 1", expect = ACCEPTABLE, desc = "actor2 -> actor1.")
@State
public class TestConcurrentIncrements {
    int v;
    @Actor public void actor1(II_Result r) {
        int tmp = v; tmp = tmp + 1; v = tmp; r.r1 = tmp;
    }
    @Actor public void actor2(II_Result r) {
        int tmp = v; tmp = tmp + 1; v = tmp; r.r2 = tmp;
    }
}
```

Homework: recording rare events

<https://github.com/Svazars/parallel-programming/tree/main/hw/block1/6.1>

Homework

Task 6.1

- *Get familiar with JCTestress*
- *Run stress test with 5 actors and single arbiter, explain results*

Stress testing

Advantages:

- Finds bugs in concurrent programs

Stress testing

Advantages:

- Finds bugs in concurrent programs

Disadvantages:

- Finds bugs in concurrent programs **probabilistically**

Finding visibility bugs

```
@Outcome(id = "0, 0", expect = ACCEPTABLE, desc = "actor2 ->actor1")
@Outcome(id = "1, 0", expect = ACCEPTABLE, desc = "actor1.x->actor2->actor1.y")
@Outcome(id = "1, 1", expect = ACCEPTABLE, desc = "actor1 ->actor2")
@Outcome(id = "0, 1", expect = ACCEPTABLE_INTERESTING, desc = "???)")
public class StrangeOrdering {
    int x; int y;
    @Actor public void actor1() {
        x = 1;
        y = 1;
    }
    @Actor public void actor2(II_Result r) {
        r.r2 = y;
        r.r1 = x;
    }
}
```

Question time

Question: Why we could observe result that does not conform to any interleaving?



Stress testing

Advantages:

- Finds bugs in concurrent programs
- Finds bugs in concurrent programs caused by compiler optimizations

Stress testing

Advantages:

- Finds bugs in concurrent programs
- Finds bugs in concurrent programs caused by compiler optimizations
 - VM internals, OS scheduling, environment specifics ...

Stress testing

Advantages:

- Finds bugs in concurrent programs
- Finds bugs in concurrent programs caused by compiler optimizations
 - VM internals, OS scheduling, environment specifics ...

Disadvantages:

- Finds bugs in concurrent programs **probabilistically**

Stress testing

Advantages:

- Finds bugs in concurrent programs
- Finds bugs in concurrent programs caused by compiler optimizations
 - VM internals, OS scheduling, environment specifics ...

Disadvantages:

- Finds bugs in concurrent programs **probabilistically**
- Finds bugs in concurrent programs run in **particular** environment

Homework: recording "impossible" events

<https://github.com/Svazars/parallel-programming/tree/main/hw/block1/6.2>

Homework

Task 6.2

- *Create stress-test equivalent to Task 1.3*
- *Find out possible concurrent results experimentally*
- *Check if all of them could be explained by interleaving model*

Concurrent reliability is hard

- **Alarm bell №1:** we do not know how to formalize *consistency*
- **Alarm bell №2:** non-determinism caused by CPU speed and scheduler decisions
- **Alarm bell №3:** concurrent problems could be OS/CPU/compiler/environment-specific

Question time

Question: Concurrency is hard! How many tests do I need to ensure my program is reliable?



Lecture plan

- 1 Concurrent invariants
- 2 Testing
 - Unit testing
 - Stress testing
 - Bug probability estimation
- 3 Tooling
- 4 Scheduling control
- 5 Bounded model checking
- 6 Summary

Validation: required mindset

"Is parallel programming hard, and, if so, what can you do about it?"(a.k.a. perfbook)⁷

Chapter 11 "Validation":

⁷ <https://github.com/paulmckrcu/perfbook>

Validation: required mindset

"Is parallel programming hard, and, if so, what can you do about it?"(a.k.a. perfbook)⁷

Chapter 11 "Validation":

- The only bug-free programs are trivial programs

⁷ <https://github.com/paulmckrcu/perfbook>

Validation: required mindset

"Is parallel programming hard, and, if so, what can you do about it?"(a.k.a. perfbook)⁷

Chapter 11 "Validation":

- The only bug-free programs are trivial programs
- A reliable program has no known bugs

⁷ <https://github.com/paulmckrcu/perfbook>

Validation: required mindset

"Is parallel programming hard, and, if so, what can you do about it?"(a.k.a. perfbook)⁷

Chapter 11 "Validation":

- The only bug-free programs are trivial programs
- A reliable program has no known bugs

Conclusion:

⁷ <https://github.com/paulmckrcu/perfbook>

Validation: required mindset

"Is parallel programming hard, and, if so, what can you do about it?"(a.k.a. perfbook)⁷

Chapter 11 "Validation":

- The only bug-free programs are trivial programs
- A reliable program has no known bugs

Conclusion:

- any reliable non-trivial program contains at least one bug that you do not know about

⁷ <https://github.com/paulmckrcu/perfbook>

Question time

Question: If a tree falls in a forest and no one is around to hear it, does it make a sound?



Question time

Question: If a program has erroneous execution trace which never happens in practice, does it count as a bug?



Enough is enough: estimate bug probability

We will discuss the simplest discrete test scenario:

- Individual test run either fails or successfully finishes⁸

⁸Continuous test example: run 1 minute under heavy contention, ensure no exceptions happen

Enough is enough: estimate bug probability

We will discuss the simplest discrete test scenario:

- Individual test run either fails or successfully finishes⁸

Probability of single failure f (e.g. 0.1, 10%)

⁸Continuous test example: run 1 minute under heavy contention, ensure no exceptions happen

Enough is enough: estimate bug probability

We will discuss the simplest discrete test scenario:

- Individual test run either fails or successfully finishes⁸

Probability of single failure f (e.g. 0.1, 10%)

Probability of single success $1 - f$ (e.g. 0.9, 90%)

⁸Continuous test example: run 1 minute under heavy contention, ensure no exceptions happen

Enough is enough: estimate bug probability

We will discuss the simplest discrete test scenario:

- Individual test run either fails or successfully finishes⁸

Probability of single failure f (e.g. 0.1, 10%)

Probability of single success $1 - f$ (e.g. 0.9, 90%)

Probability n repeats will succeed $S_n = (1 - f)^n$

⁸Continuous test example: run 1 minute under heavy contention, ensure no exception happen

Enough is enough: estimate bug probability

We will discuss the simplest discrete test scenario:

- Individual test run either fails or successfully finishes⁸

Probability of single failure f (e.g. 0.1, 10%)

Probability of single success $1 - f$ (e.g. 0.9, 90%)

Probability n repeats will succeed $S_n = (1 - f)^n$

Probability of failure $F_n = 1 - (1 - f)^n$

⁸Continuous test example: run 1 minute under heavy contention, ensure no exception happen

Enough is enough: estimate bug probability

We will discuss the simplest discrete test scenario:

- Individual test run either fails or successfully finishes⁸

Probability of single failure f (e.g. 0.1, 10%)

Probability of single success $1 - f$ (e.g. 0.9, 90%)

Probability n repeats will succeed $S_n = (1 - f)^n$

Probability of failure $F_n = 1 - (1 - f)^n$

"How many times should we run the test to cause the probability of failure to rise above 99%?"

⁸Continuous test example: run 1 minute under heavy contention, ensure no exceptions happen

Enough is enough: estimate bug probability

We will discuss the simplest discrete test scenario:

- Individual test run either fails or successfully finishes⁸

Probability of single failure f (e.g. 0.1, 10%)

Probability of single success $1 - f$ (e.g. 0.9, 90%)

Probability n repeats will succeed $S_n = (1 - f)^n$

Probability of failure $F_n = 1 - (1 - f)^n$

"How many times should we run the test to cause the probability of failure to rise above 99%?"

$$n = \frac{\log(1 - F_n)}{\log(1 - f)}$$

⁸Continuous test example: run 1 minute under heavy contention, ensure no exceptions happen

Be careful. It is real world out there.

- As of 2017 Linux kernel was estimated to have more than 20 billion instances running
- Bug that occurs once every million years on a single system will be encountered more than

Be careful. It is real world out there.

- As of 2017 Linux kernel was estimated to have more than 20 billion instances running
- Bug that occurs once every million years on a single system will be encountered more than
 - 50 times per day

Be careful. It is real world out there.

- As of 2017 Linux kernel was estimated to have more than 20 billion instances running
- Bug that occurs once every million years on a single system will be encountered more than
 - 50 times per day

Any mission-critical software for

- health⁹
- aerospace¹⁰
- military¹¹

is also not satisfied with "just testing".

⁹ <https://en.wikipedia.org/wiki/Therac-25>

¹⁰ https://en.wikipedia.org/wiki/Ariane_flight_V88

¹¹ <https://devblogs.microsoft.com/oldnewthing/20180228-00/?p=98125>

Concurrent reliability is hard

- **Alarm bell №1:** we do not know how to formalize *consistency*
- **Alarm bell №2:** non-determinism caused by CPU speed and scheduler decisions
- **Alarm bell №3:** concurrent problems could be OS/CPU/compiler/environment-specific
- **Alarm bell №4:** decreasing concurrent bug probability requires excessive computing power
- **Alarm bell №5:** we do not know how to guarantee "zero concurrent bug" property

Question time

Question: There are a lot of programs that operate on other programs: compilers, linters, debuggers ... Any help with prevention of concurrent bugs?



Lecture plan

- 1 Concurrent invariants
- 2 Testing
 - Unit testing
 - Stress testing
 - Bug probability estimation
- 3 Tooling
- 4 Scheduling control
- 5 Bounded model checking
- 6 Summary

Static code analysis

- Java FindBugs <https://findbugs.sourceforge.net>
 - <https://findbugs.sourceforge.net/bugDescriptions.html>, search for "synchronized" or "concurrent"
 - "Finding Concurrency Bugs In Java" by David Hovemeyer and William Pugh¹²
- IneliJ IDEA inspections and analysis, interactive debugger¹³
- "Java Concurrency in Practice"(JCIP)
<https://jcip.net/annotations/doc/index.html>

¹² <https://www.cs.jhu.edu/~daveho/pubs/csjs2004.pdf>

¹³ <https://www.jetbrains.com/help/idea/detect-concurrency-issues.html>

Static code analysis

- Java FindBugs <https://findbugs.sourceforge.net>
 - <https://findbugs.sourceforge.net/bugDescriptions.html>, search for "synchronized" or "concurrent"
 - "Finding Concurrency Bugs In Java" by David Hovemeyer and William Pugh¹²
- IntelliJ IDEA inspections and analysis, interactive debugger¹³
- "Java Concurrency in Practice"(JCIP)
<https://jcip.net/annotations/doc/index.html>

Alarm bell №6: static analysis lack run-time knowledge (false positives)

¹² <https://www.cs.jhu.edu/~daveho/pubs/csjp2004.pdf>

¹³ <https://www.jetbrains.com/help/idea/detect-concurrency-issues.html>

Dynamic checks

- Assertions
- Configurable heavyweight checks
- Compile-time program rewriting¹⁴
- Run-time program transformation¹⁵

¹⁴ https://en.wikipedia.org/wiki/Aspect-oriented_programming

¹⁵ <https://docs.oracle.com/en/java/javase/11/docs/api/java.instrument/java/lang/instrument/package-summary.html>

Dynamic checks

- Assertions
- Configurable heavyweight checks
- Compile-time program rewriting¹⁴
- Run-time program transformation¹⁵

Other languages also support dynamic checks, popular term is "sanitizers" or "race detectors":

- <https://valgrind.org>
- https://go.dev/doc/articles/race_detector
- <https://docs.kernel.org/dev-tools/ubsan.html>
- <https://clang.llvm.org/docs/AddressSanitizer.html>

¹⁴ https://en.wikipedia.org/wiki/Aspect-oriented_programming

¹⁵ <https://docs.oracle.com/en/java/javase/11/docs/api/java.instrument/java/lang/instrument/package-summary.html>

Concurrent reliability is hard

- **Alarm bell №1:** we do not know how to formalize *consistency*
- **Alarm bell №2:** non-determinism caused by CPU speed and scheduler decisions
- **Alarm bell №3:** concurrent problems could be OS/CPU/compiler/environment-specific
- **Alarm bell №4:** decreasing concurrent bug probability requires excessive computing power
- **Alarm bell №5:** we do not know how to guarantee "zero concurrent bug" property
- **Alarm bell №6:** static analysis lack run-time knowledge (false positives)
- **Alarm bell №7:** dynamic analysis works for particular execution trace only

Concurrent reliability is hard

- **Alarm bell №1:** we do not know how to formalize *consistency*
- **Alarm bell №2:** non-determinism caused by CPU speed and scheduler decisions
- **Alarm bell №3:** concurrent problems could be OS/CPU/compiler/environment-specific
- **Alarm bell №4:** decreasing concurrent bug probability requires excessive computing power
- **Alarm bell №5:** we do not know how to guarantee "zero concurrent bug" property
- **Alarm bell №6:** static analysis lack run-time knowledge (false positives)
- **Alarm bell №7:** dynamic analysis works for particular execution trace only

Reminder (lecture 1): non-determinism is our main enemy through the course

Lecture plan

- 1 Concurrent invariants
- 2 Testing
 - Unit testing
 - Stress testing
 - Bug probability estimation
- 3 Tooling
- 4 Scheduling control
- 5 Bounded model checking
- 6 Summary

Need for speed

Problem:

- Unpredictable speed of execution

Need for speed

Problem:

- Unpredictable speed of execution

Consequences:

- Hard to detect bugs
- Hard to reproduce problems
- Hard to verify issue is fixed

Question time

Question: Could we change timings of multi-threaded program?



Scheduling control: language level

Custom scheduling using language primitives:

- Arbitrary `Thread.sleep` here and there

Scheduling control: language level

Custom scheduling using language primitives:

- Arbitrary `Thread.sleep` here and there
- Guided insertion of randomized delay for synchronization operations
 - Efficient Scalable Thread-Safety-Violation Detection¹⁶

¹⁶<https://rohan.padhye.org/files/tsvd-sosp19.pdf>

Scheduling control: language level

Custom scheduling using language primitives:

- Arbitrary `Thread.sleep` here and there
- Guided insertion of randomized delay for synchronization operations
 - Efficient Scalable Thread-Safety-Violation Detection¹⁶
- Analysis of communication points (memory locations)
 - Snowboard: Finding Kernel Concurrency Bugs through Systematic Inter-thread Communication Analysis¹⁷

¹⁶ <https://rohan.padhye.org/files/tsvd-sosp19.pdf>

¹⁷ <https://sishuaigong.github.io/pdf/sosp21-snowboard.pdf>

Scheduling control: language level

Custom scheduling using language primitives:

- Arbitrary `Thread.sleep` here and there
- Guided insertion of randomized delay for synchronization operations
 - Efficient Scalable Thread-Safety-Violation Detection¹⁶
- Analysis of communication points (memory locations)
 - Snowboard: Finding Kernel Concurrency Bugs through Systematic Inter-thread Communication Analysis¹⁷

Alarm bell №3: concurrent problems could be OS/CPU/compiler/environment-specific

¹⁶ <https://rohan.padhye.org/files/tsvd-sosp19.pdf>

¹⁷ <https://sishuaigong.github.io/pdf/sosp21-snowboard.pdf>

Scheduling control: OS level

OS scheduler:

- Scheduling policy (round-robin, shortest remaining time ...)
- Thread priorities, Scheduling quantum, Context switch, Fairness, Real-time

Scheduling control: OS level

OS scheduler:

- Scheduling policy (round-robin, shortest remaining time ...)
- Thread priorities, Scheduling quantum, Context switch, Fairness, Real-time

Pinning/affinity¹⁸:

- Thread A could be executed only by Processor 1 or Processor 7
- `taskset`¹⁹, `numactl`²⁰

¹⁸ https://en.wikipedia.org/wiki/Processor_affinity

¹⁹ <https://man7.org/linux/man-pages/man1/taskset.1.html>

²⁰ <https://linux.die.net/man/8/numactl>

Scheduling control: OS level

OS scheduler:

- Scheduling policy (round-robin, shortest remaining time ...)
- Thread priorities, Scheduling quantum, Context switch, Fairness, Real-time

Pinning/affinity:

- Thread A could be executed only by Processor 1 or Processor 7
- `taskset`, `numactl`

Scheduling control: OS level

OS scheduler:

- Scheduling policy (round-robin, shortest remaining time ...)
- Thread priorities, Scheduling quantum, Context switch, Fairness, Real-time

Pinning/affinity:

- Thread A could be executed only by Processor 1 or Processor 7
- `taskset`, `numactl`

Replace/record OS scheduling decisions to enable deterministic execution:

- Reproducible container²¹
- Lightweight record-replay²²
- Time-travel debugging²³

²¹<https://github.com/facebookexperimental/hermit>

²²<https://rr-project.org/>

²³https://en.wikipedia.org/wiki/Time_travel_debugging

Question time

Question: How did you managed to solve

- Task 1.2 (Prove main thread may not observe $x == 1$, $r_y == 0$, $r_z == 1$)?
- Task 1.3 (Provide all possible execution results and prove exhaustiveness)?



Lecture plan

- 1 Concurrent invariants
- 2 Testing
 - Unit testing
 - Stress testing
 - Bug probability estimation
- 3 Tooling
- 4 Scheduling control
- 5 Bounded model checking**
- 6 Summary

Interleaving model

N:1 threading model

- N preemptible tasks, Single executor

Preemption could happen

- At any place (any machine instruction)
- At any time (every femtosecond)
- For arbitrary long time (undefined scheduling policy)

Executed instructions instantly affect whole computing system

- All subsequent instructions see up-to-date memory state
- All threads observe the same up-to-date memory state
- All events are totally ordered on a timeline

Non-determinism:

- Concurrent program may have **many** execution traces.
- Concurrent program may have **many** results of execution.

Interleaving model

- N:1 threading model, single executor
- Preemption could happen anywhere
- All events are totally ordered on a timeline

Interleaving model

- N:1 threading model, single executor
- Preemption could happen anywhere
- All events are totally ordered on a timeline

Looks like a lot of mundane automatic work for computer program!

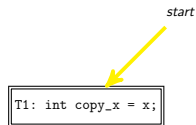
State space exploration

start

```
Thread 1  
int copy_x = x;  
x++;
```

```
Thread 2  
int copy_x = x;  
x++;
```

State space exploration



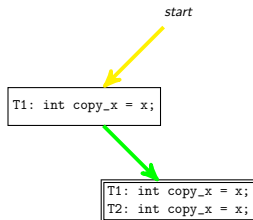
Thread 1

```
int copy_x = x;  
x++;
```

Thread 2

```
int copy_x = x;  
x++;
```

State space exploration



Thread 1

```
int copy_x = x;
```

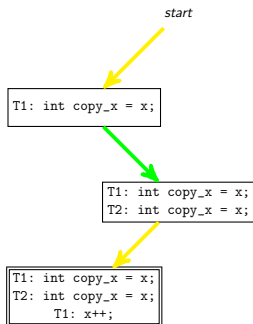
```
x++;
```

Thread 2

```
int copy_x = x;
```

```
x++;
```

State space exploration



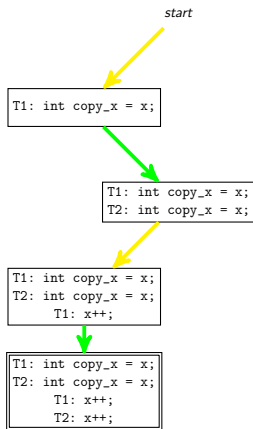
Thread 1

```
int copy_x = x;
x++;
```

Thread 2

```
int copy_x = x;
x++;
```

State space exploration



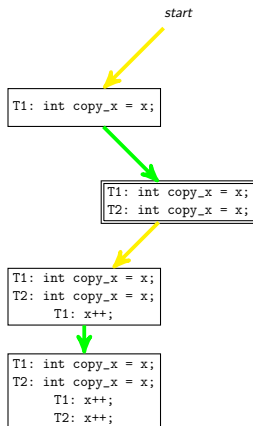
Thread 1

```
int copy_x = x;
x++;
```

Thread 2

```
int copy_x = x;
x++;
```

State space exploration



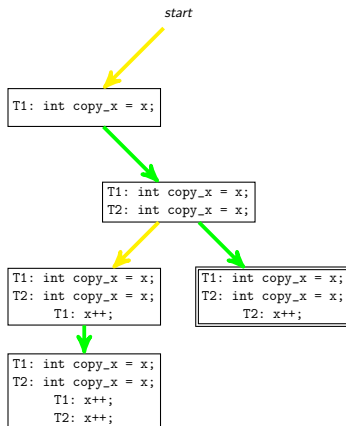
Thread 1

```
int copy_x = x;
x++;
```

Thread 2

```
int copy_x = x;
x++;
```

State space exploration



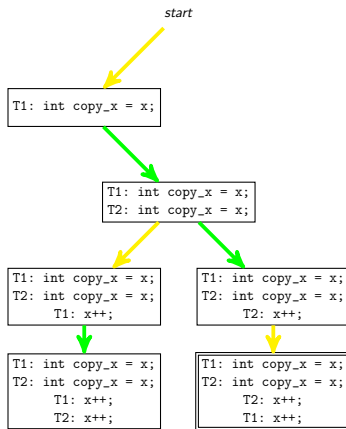
Thread 1

```
int copy_x = x;
x++;
```

Thread 2

```
int copy_x = x;
x++;
```

State space exploration



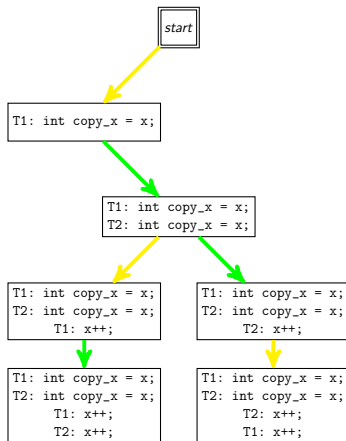
Thread 1

```
int copy_x = x;
x++;
```

Thread 2

```
int copy_x = x;
x++;
```

State space exploration



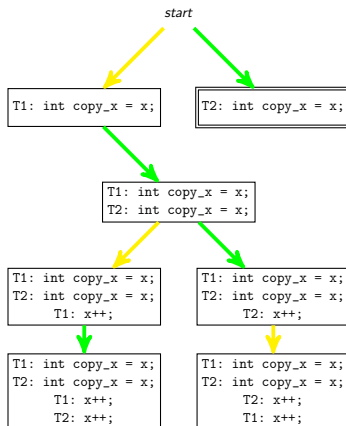
Thread 1

```
int copy_x = x;
x++;
```

Thread 2

```
int copy_x = x;
x++;
```

State space exploration



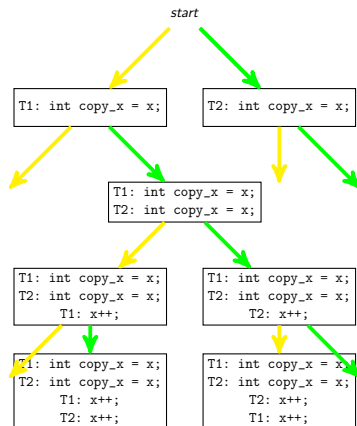
Thread 1

```
int copy_x = x;
x++;
```

Thread 2

```
int copy_x = x;
x++;
```

State space exploration



Thread 1

```
int copy_x = x;
x++;
```

Thread 2

```
int copy_x = x;
x++;
```

Model checking

Idea:

- **Enumerate** all possible execution traces
- **Analyze** each execution trace for defects

Model checking

Idea:

- **Enumerate** all possible execution traces
- **Analyze** each execution trace for defects

Challenges:

- Unlimited state space of programs with loops/recursion
- Many threads/instructions == combinatorial explosion of reachable states

Model checking

Idea:

- **Enumerate** all possible execution traces
- **Analyze** each execution trace for defects

Challenges:

- Unlimited state space of programs with loops/recursion
- Many threads/instructions == combinatorial explosion of reachable states

Computer Scientists have already thought about this?

Question time

Question: Which domains require **very** reliable software?



Java Path Finder

- Extensible model checking framework for Java bytecode programs

Java Path Finder

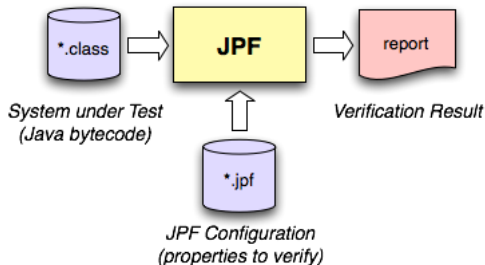
- Extensible model checking framework for Java bytecode programs
- Developed at the NASA Ames Research Center. Open sourced in 2005 (Apache-2.0)
- <https://github.com/javapathfinder/jpf-core>

Java Path Finder

- Extensible model checking framework for Java bytecode programs
- Developed at the NASA Ames Research Center. Open sourced in 2005 (Apache-2.0)
- <https://github.com/javapathfinder/jpf-core>
- Effectively – research JVM written in Java

Java Path Finder

- Extensible model checking framework for Java bytecode programs
- Developed at the NASA Ames Research Center. Open sourced in 2005 (Apache-2.0)
- <https://github.com/javapathfinder/jpf-core>
- Effectively – research JVM written in Java



We are not going to Mars literally, aren't we?

Framework from NASA Ames Research Center.

We are not going to Mars literally, aren't we?

Framework from NASA Ames Research Center.

https://www.nasa.gov/centers/ames/news/releases/2005/05_28AR.html

JPF was used to detect inconsistencies in the executive software for the K9 Rover at NASA Ames. The K9 rover is a six-wheeled, solar-powered rover developed jointly at NASA Ames and NASA's Jet Propulsion Laboratory



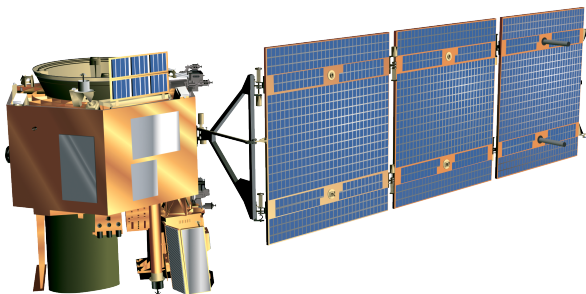
We are not going to Mars literally, aren't we?

Framework from NASA Ames Research Center.

https://www.nasa.gov/centers/ames/news/releases/2005/05_28AR.html

https://en.wikipedia.org/wiki/Earth_Observing-1

... scientists used elements of JPF to develop verification computer code for Livingstone 2 software, a diagnosis system now flying on the EO-1 spacecraft.



Homework: rocket science

<https://github.com/Svazars/parallel-programming/tree/main/hw/block1/6.3>

Homework

Task 6.3

- *Get familiar with JPF, Task 1.2 (almost) solved automatically*
- *Finish it!*

<https://github.com/Svazars/parallel-programming/tree/main/hw/block1/6.4>

Homework

Task 6.4

- *Solve Task 1.3 using JPF*

Concurrent reliability is hard

- **Alarm bell №1:** we do not know how to formalize *consistency*
- **Alarm bell №2:** non-determinism caused by CPU speed and scheduler decisions
- **Alarm bell №3:** concurrent problems could be OS/CPU/compiler/environment-specific
- **Alarm bell №4:** decreasing concurrent bug probability requires excessive computing power
- **Alarm bell №5:** we do not know how to guarantee "zero concurrent bug" property
- **Alarm bell №6:** static analysis lack run-time knowledge (false positives)
- **Alarm bell №7:** dynamic analysis works for particular execution trace only

Concurrent reliability is hard?

- **Alarm bell №1:** we do not know how to formalize *consistency*
- **Alarm bell №3:** concurrent problems could be OS/CPU/compiler/environment-specific

Automation of automation

https://en.wikipedia.org/wiki/Clarke%27s_three_laws

Any sufficiently advanced technology is indistinguishable from magic

Automation of automation

https://en.wikipedia.org/wiki/Clarke%27s_three_laws

Any sufficiently advanced technology is indistinguishable from magic

- Tools that automatically generate concurrent tests for my classes?

Automation of automation

https://en.wikipedia.org/wiki/Clarke%27s_three_laws

Any sufficiently advanced technology is indistinguishable from magic

- Tools that automatically generate **reliable** concurrent tests for my classes?

Automation of automation

https://en.wikipedia.org/wiki/Clarke%27s_three_laws

Any sufficiently advanced technology is indistinguishable from magic

- Tools that automatically generate **reliable** concurrent tests for my classes?
- Tools that find visibility bugs without running various software/hardware configurations?

Question time

Question: Do you believe in Compute Science magic?



Supernatural tools: lincheck

<https://kotlinlang.org/docs/lincheck-guide.html>

- Creates **concurrent** tests for your data structure
- Runs multithreaded stress-tests
- Explores possible interleavings via bounded model checking

Supernatural tools: lincheck

<https://kotlinlang.org/docs/lincheck-guide.html>

- Creates **concurrent** tests for your data structure
- Runs multithreaded stress-tests
- Explores possible interleavings via bounded model checking

Verifies that the results of each invocation satisfy the required correctness property (linearizability is the default one).

Supernatural tools: lincheck

<https://kotlinlang.org/docs/lincheck-guide.html>

- Creates **concurrent** tests for your data structure
- Runs multithreaded stress-tests
- Explores possible interleavings via bounded model checking

Verifies that the results of each invocation satisfy the required correctness property (linearizability is the default one).

<https://kotlinlang.org/docs/progress-guarantees.html>

... wait-freedom ... lock-freedom ... obstruction-freedom

Supernatural tools: lincheck

<https://kotlinlang.org/docs/lincheck-guide.html>

- Creates **concurrent** tests for your data structure
- Runs multithreaded stress-tests
- Explores possible interleavings via bounded model checking

Verifies that the results of each invocation satisfy the required correctness property (linearizability is the default one).

<https://kotlinlang.org/docs/progress-guarantees.html>

... wait-freedom ... lock-freedom ... obstruction-freedom

<https://kotlinlang.org/docs/sequential-specification.html>

To be sure that the algorithm provides correct sequential behavior, you can define its sequential specification

Concurrent reliability is hard?

- **Alarm bell №1:** we do not know how to formalize *consistency*
- **Alarm bell №3:** concurrent problems could be OS/CPU/compiler/environment-specific

Supernatural tools: herd

<https://diy.inria.fr/www/>

AArch64 MP

```
-----
P0          | P1          ;
MOV W0,#1    | LDR W0,[y]   ;
STR W0,[x]   | LDR W2,[x]   ;
MOV W2,#1    |              ;
STR W2,[y]   |              ;
```

exists

(P1:W0=1 /\ P1:W2=0)

Supernatural tools: herd

<https://diy.inria.fr/www/>

AArch64 MP

```
-----
P0          | P1          ;
MOV W0,#1   | LDR W0,[y]   ;
STR W0,[x]  | LDR W2,[x]   ;
MOV W2,#1   |              ;
STR W2,[y]  |              ;
```

exists

(P1:W0=1 /\ P1:W2=0)

How he knows which loads and stores AArch64 CPU is allowed to reorder?

Supernatural tools: herd

<https://diy.inria.fr/www/>

AArch64 MP

```
-----
P0          | P1          ;
MOV W0,#1   | LDR W0,[y]   ;
STR W0,[x]  | LDR W2,[x]   ;
MOV W2,#1   |              ;
STR W2,[y]  |              ;
```

exists

(P1:W0=1 /\ P1:W2=0)

How he knows which loads and stores AArch64 CPU is allowed to reorder?

Such analysis could be run even in *my browser*?

Supernatural tools: herd

<https://github.com/herd/herdtools7>

- generic simulator for weak memory models
- produce litmus tests from concise specifications
- run litmus tests to test the memory model of the executing machine

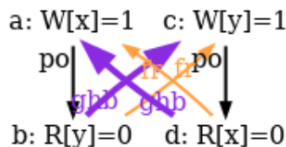
Supernatural tools: herd

<https://github.com/herd/herdtools7>

- generic simulator for weak memory models
- produce litmus tests from concise specifications
- run litmus tests to test the memory model of the executing machine

<https://diy.inria.fr/newdoc/herd.html>

The non-SC behaviour is now accepted, as write-to-read po-pairs do not participate to the acyclicity check any more. In effect, this allows the last execution above, as ghb (i.e. po-tso / com-tso) is acyclic.



Supernatural tools: to use or not to use?

Concurrency-targeted tools could do all the heavy lifting

- Create concurrent tests for specified data structure
- Enumerate all possible interleavings for particular test
 - show execution trace for any found defect
- Reliably detect weak memory model quirks
 - no need to buy arm32+aarch64+power CPUs to reproduce

Supernatural tools: to use or not to use?

Concurrency-targeted tools could do all the heavy lifting

- Create concurrent tests for specified data structure
- Enumerate all possible interleavings for particular test
 - show execution trace for any found defect
- Reliably detect weak memory model quirks
 - no need to buy arm32+aarch64+power CPUs to reproduce

Such tools expect you to have broad context

- concurrent consistency, sequential specification, linearizability
- read-modify-write, wait-free, lock-free, obstruction-free
- cache coherency, weak memory model, memory barriers, litmus tests
- visibility, language memory model

Supernatural tools: to use or not to use?

Concurrency-targeted tools could do all the heavy lifting

- Create concurrent tests for specified data structure
- Enumerate all possible interleavings for particular test
 - show execution trace for any found defect
- Reliably detect weak memory model quirks
 - no need to buy arm32+aarch64+power CPUs to reproduce

Such tools expect you to have broad context

- concurrent consistency, sequential specification, linearizability
- read-modify-write, wait-free, lock-free, obstruction-free
- cache coherency, weak memory model, memory barriers, litmus tests
- visibility, language memory model

Looks like a plan for second block of this course!

Reliable concurrent software: chasing the horizon

Any error must manifest itself as soon as possible:

- Readable and complete documentation, locking policy, inheritance suggestions
- External invariants (exceptions) + internal invariants (assertions)

Use as many validation techniques as you could:

- Unit testing
- Stress testing
- Estimate bug propability and required post-fix testing effort

Master tools:

- Test generation
- Static checks
- Dynamic checks
- Scheduling randomization
- Bounded model checking

Summary: homework

Lecture 6, Tasks 6.*: <https://github.com/Svazars/parallel-programming/blob/main/hw/block1>